LangChain vs LangGraph - Detailed Study Notes

CampusX • Agentic AI using LangGraph • Video 3 • 1 h 27 min 29 sec
Original video: https://www.youtube.com/watch?v=31qyMKNB2RA

At a glance

Main idea: LangChain gives you convenient building blocks for LLM-powered applications. LangGraph lets you **orchestrate complex, stateful, non-linear and potentially long-running workflows** using nodes, edges, persistent state and checkpoints. In practice, you can use both together.

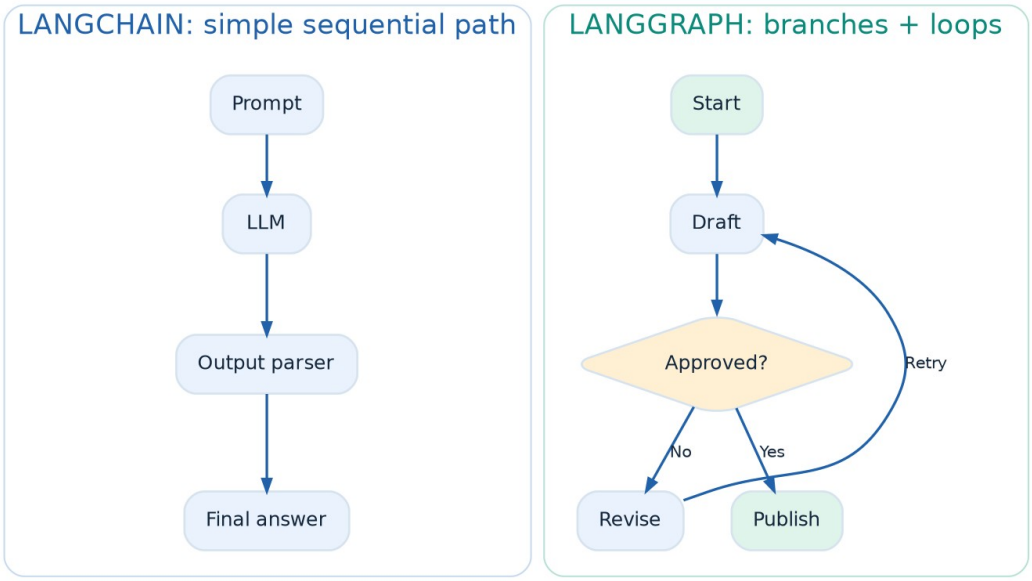


Figure 1. Linear chain compared with a graph

A 60-second mental model

Think of **LangChain as the toolkit** (model calls, prompt templates, retrievers and tools) and **LangGraph as the workflow engine** (what to run, what to remember and where to go next). You can combine them rather than choosing one exclusively.

Imagine a cooking application. LangChain provides ingredients and kitchen tools; LangGraph is the recipe with branches such as *if the cake fails the taste test, adjust and bake again*. The recipe also remembers whether the batter is prepared and whether a chef has approved the result.

Two questions to ask about any AI application:

1. *What capabilities are needed?* For example retrieval, prompting, tool calling or structured output. These are LangChain strengths.
2. *How should execution be controlled?* For example routing, retries, state, approvals, and resuming later. These are LangGraph strengths.

Reading tip: follow the hiring example through every comparison. It is easier to learn one scenario deeply than seven isolated definitions.

Video chapter guide

Time	Topic	Why it matters
00:00–02:30	Introduction and goal	Why compare the two frameworks?
05:20	What is LangChain?	Building blocks for LLM apps
10:15	LangChain use cases	Where simple chains are enough
15:40	Automated workflow	Hiring example motivates the need for orchestration
27:36	Control-flow complexity	Branches, loops and routes
39:16	Handling state	Keeping application data across steps
49:50	Event-driven execution	Pauses and external events
55:55	Fault tolerance	Resume after interruptions or failures
1:02:26	Human-in-the-loop	Approval and feedback
1:08:54	Nested workflows	Subgraphs and reuse
1:15:53	Observability	Trace, debug and inspect execution
1:22:38	Revision	When to choose which framework

1. What is LangChain? - 05:20

Definition: LangChain is a library for composing applications around language models. Instead of writing every provider/API integration yourself, you use reusable components.

- **Models:** Standardized interfaces to chat models and other model providers.
- **Prompt templates:** Build structured, reusable model inputs.
- **Retrievers and vector stores:** Retrieve relevant documents for RAG.
- **Tools:** Let a model interact with APIs or Python functions.
- **Chains / runnables:** Connect operations, e.g., `prompt → LLM → parser`.

When it shines: Small chatbots, document Q&A, summarization, classification, simple RAG, or a predictable sequence of calls.

Important distinction: The lesson contrasts simple *chains* with advanced graph orchestration. That does **not** mean every LangChain app is incapable of branching, tool use, or memory. More recent LangChain agents themselves use LangGraph internally.

Practical example: the same RAG task at two complexity levels

Version A - simple Q&A: retrieve the three most relevant PDF chunks, put them into a prompt, ask an LLM for an answer, and return the result. The route is known in advance: `Question → Retriever → Prompt → Model → Answer`. A short LangChain pipeline may be sufficient.

Version B - research assistant: classify the question, decide whether to search PDFs or the web, evaluate the evidence, request more evidence if confidence is low, ask for approval before emailing a report, and store progress across sessions. A graph makes the **decisions and loops** explicit.

Do not confuse model reasoning with orchestration. An LLM may suggest what to do, but application code must still handle tool permissions, state changes, stopping conditions and failures.

2. The worked example: automate hiring - 15:40

The video uses an **automated recruitment process** to show why a real application is more complicated than `Input → LLM → Output`.

1. A hiring manager requests a new position.
2. An LLM generates a **job description (JD)**.
3. A human reviews the JD; if rejected, **revise and loop**.
4. The approved JD is posted to job platforms.
5. The process waits for applications (possibly for days).
6. Evaluate applications; if response is too low, modify/repost the JD and repeat.
7. Parse resumes and rank candidates.
8. Schedule interviews via calendars and messages.
9. Collect decisions, make offers, and negotiate or revisit candidates as required.
10. Begin onboarding when the offer is accepted.

What makes it agentic/workflow-like? Conditional decisions, loops, a shared record of candidates and decisions, external APIs, long pauses, human approvals, failures, and nested subprocesses.

Trace one request from start to finish

Step	Example data in state	Possible next step
Draft JD	`jd="Python backend engineer..."`	Human review
Recruiter rejects	`approved=False`, `feedback="add AWS"`	Rewrite JD
Recruiter approves	`approved=True`	Publish job
Publishing succeeds	`status="posted"`, `job_id="JOB-42"`	Wait for applicants
Applications arrive	`applicants=[...]`	Screening
Candidate selected	`shortlist=[...]`	Interview / offer

The workflow **does not start over** when the recruiter rejects the JD. It follows a loop back to the relevant step, carrying the feedback in shared state. When waiting for applications, a separately configured webhook or job runner can resume the saved thread.

Mapping the example to the seven features:

Recruiting step	What the system needs
Draft, then revise JD until approved	Conditional edges and loops
Store latest JD, applicants and feedback	Shared typed state
Wait for applications or responses	Event-driven, resumable execution

Recruiting step	What the system needs
Retry a failing posting or calendar API	Checkpoints, retries and safe recovery
Ask recruiter to approve or modify JD	Human-in-the-loop interrupt
Run resume-screening and interview modules	Reusable subgraphs
Find why a candidate was rejected	Tracing and observability

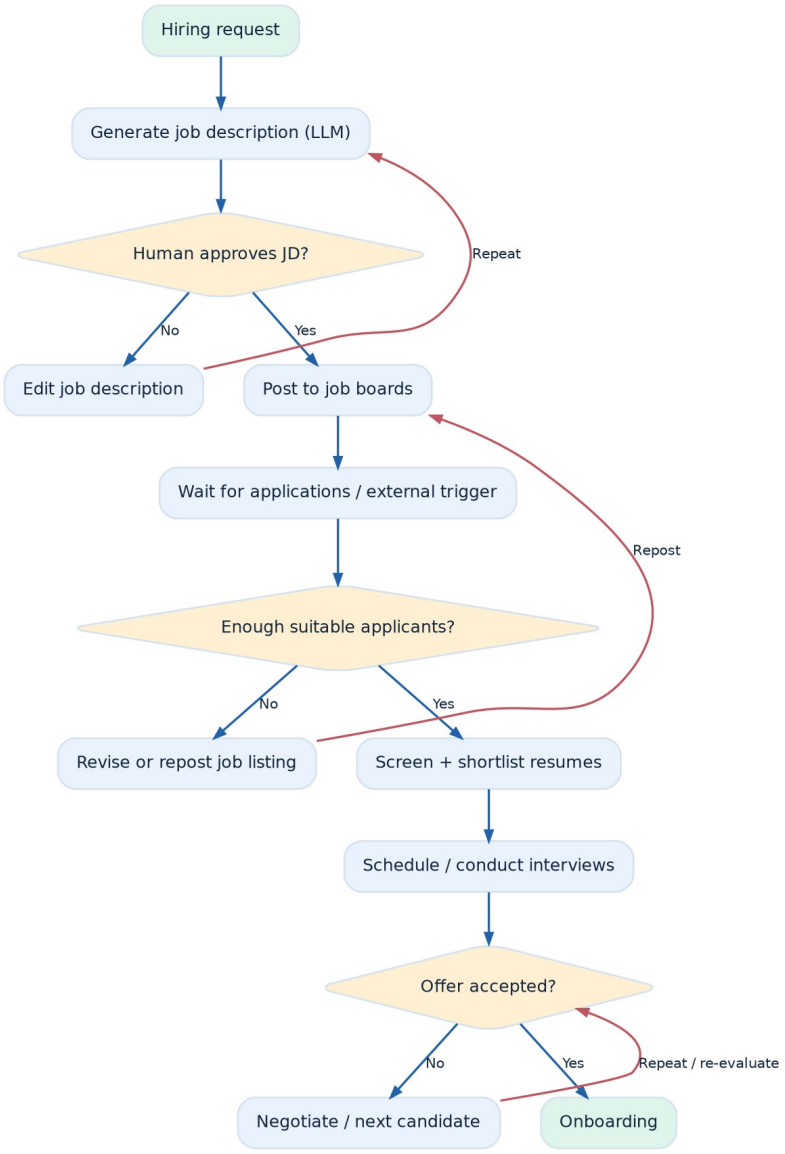


Figure 2. Hiring workflow with loops and human decisions

Ask while studying: If a process is waiting for a recruiter for three days, how will it remember the JD, applicants and the exact step to resume?

3. Seven comparisons explained

3.1 Control-flow complexity - 27:36

A **linear chain** has a mostly fixed next step. Real workflows need `if/else` branches, retries, feedback loops and decisions that revisit earlier steps. You can implement these with Python around LangChain components, but extensive custom orchestration becomes harder to read, test and debug.

LangGraph approach: Represent actions as **nodes**, routes as **edges**, and choices as **conditional edges**. Loops are normal graph paths.

- Example: `Draft JD → Check approval → (reject → Revise → Check again) OR (approve → Publish).`
- **Takeaway:** Prefer a graph when the control path itself is a central part of the application.

How a conditional decision is represented: the review step might produce `"revise"` when approval is false and `"publish"` otherwise. A conditional edge chooses the next node. The graph's structure stays readable even as the decision logic grows.

Design rule: let each node do one clear job. For instance, `draft_jd`, `review_jd`, and `publish_jd` should be separate rather than packing all three into one opaque 500-line function. Give feedback loops a maximum attempt count or an escalation path to avoid endless retries.

Try it: draw `Draft → Review` and label two outgoing paths: `approved → Publish` and `rejected → Revise → Review`.

3.2 State management - 39:16

State is the evolving information a workflow needs: job description, hiring status, applicant list, approval decision, interview results and so on. When dozens of steps depend on it, manually passing values and updating global dictionaries becomes fragile.

LangGraph approach: Define a typed **state schema** (commonly `TypedDict`), then have each node **return updates** for the relevant state keys. The graph carries state between nodes. For keys updated by multiple nodes, reducers can determine how updates combine.

Example state (conceptual):

```
class HiringState(TypedDict):
    job_description: str
    approved: bool
    applicants: list[str]
    selected_candidate: str | None
```



Figure 3. Shared state and checkpoints

Remember: State is the current working data. A **checkpoint** is a saved snapshot of that state at an execution boundary, so a thread can later be inspected or resumed.

State, checkpoint and store - three different things

- **State:** the current working values for a running thread, such as `jd`, `approved`, and `applicants`.
- **Checkpoint:** a saved snapshot of a thread's state and execution progress. It helps inspect or resume the workflow.
- **Store:** optional application-level persistent data that may be accessed across threads, such as a reusable organization preference.

Concrete update: suppose state initially contains `{"jd": "Draft A", "approved": False}`. A reviewer node returns `{"approved": True}`; the resulting state still contains the JD. For fields with no explicit reducer, a later write normally replaces the prior value. List merging is different; configure a reducer if updates need to accumulate.

Subtle but important: a state schema does not automatically make data durable. For cross-process crash recovery, configure a durable checkpointer (not just an in-memory saver).

3.3 Event-driven and long-running execution - 49:50

A business workflow can wait for a form submission, email reply, approval, a webhook or a scheduled date. Keeping one Python process blocked for days is generally not a sound approach.

LangGraph approach: Use **durable state and resumable execution** so the process can stop and later continue when an external trigger arrives. Use an external scheduler, queue, webhook or application service to determine *when* to resume; LangGraph is not by itself a calendar/scheduling service.

- Example: Publish the JD now; resume candidate evaluation after applications arrive.
- **Takeaway:** Separate execution progress from the lifetime of a single process.

What actually triggers the resume? The graph can be paused and persisted; another part of your system receives a webhook, email event, approval action or scheduled job, loads the correct `thread_id`, and invokes the graph again. This separation is essential for systems that may wait hours or days.

Example timeline: Monday 09:00 - JD posted and workflow paused. Wednesday 14:00 - application webhook arrives. Wednesday 14:01 - backend resumes `hiring-42` using the new applicants. The graph continues from saved progress rather than relying on one always-running HTTP request.

3.4 Fault tolerance - 55:55

If the interview-scheduling API fails after resumes are screened, you do not want to restart the entire hiring process blindly.

LangGraph approach: A configured **checkpointer** records execution state; an interrupted workflow can use saved progress. Retry policies can address transient failures, and application design still needs safe/idempotent external operations.

- **Checkpoint:** Saved state at a boundary.
- **Retry:** Attempt a failing operation again according to policy.
- **Recovery:** Resume appropriately after an error or interruption.

Production caveat: In-memory persistence does not survive process restarts. A durable checkpointer is needed for real crash recovery.

Where retries are risky: a failed LLM call can often be retried, but a payment, offer email or job posting may have taken effect *before* the network timed out. Replaying an action carelessly can send duplicates. Use idempotency keys, external-system IDs and deduplication; make side effects safe to repeat.

Failure example: resume at `schedule_interview` after a calendar API timeout. First check whether the event already exists for that candidate. Only create a new event if it does not. Checkpoints preserve workflow progress, but do not automatically make external operations transactional.

3.5 Human-in-the-loop (HITL) - 1:02:26

Certain decisions should be reviewed by a person: approve a JD, reject a candidate, confirm a compensation offer or authorize an action.

LangGraph approach: A node can **interrupt** execution, save progress, present the required input and then resume that same thread with a human response.

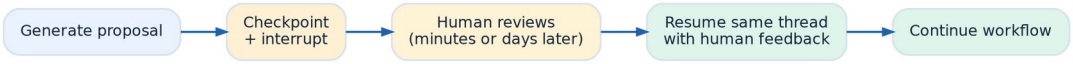


Figure 4. Human-in-the-loop approval

Key idea: `Generate → PAUSE → Human decision → RESUME → Continue`. The human can respond much later without requiring the same synchronous request to remain open.

A real human approval in Python

The pattern below demonstrates the **pause and resume mechanism**. The state is stored in memory for local learning; a persistent checkpointer is needed across application restarts.

```
from typing import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import interrupt, Command

class ApprovalState(TypedDict):
    draft: str
    approved: bool

def ask_recruiter(state: ApprovalState):
    decision = interrupt({"question": "Approve this JD?", "draft": state["draft"]})
    return {"approved": bool(decision)}

builder = StateGraph(ApprovalState)
builder.add_node("review", ask_recruiter)
builder.add_edge(START, "review")
builder.add_edge("review", END)
app = builder.compile(checkpointer=InMemorySaver())
config = {"configurable": {"thread_id": "hiring-demo-1"}}

paused = app.invoke({"draft": "Python engineer", "approved": False}, config)
# A UI would show the interrupt payload and collect a human decision.
finished = app.invoke(Command(resume=True), config)
print(finished["approved"])  # True
```

Important execution detail: the node can re-run from its beginning on resume. Code before `interrupt()` should therefore avoid non-idempotent side effects such as sending an email or charging a card.

3.6 Nested workflows / subgraphs - 1:08:54

Instead of one enormous hiring graph, encapsulate activities as smaller reusable graphs.

- **JD subgraph:** Generate → Review → Revise.
- **Resume screening subgraph:** Parse → Score → Shortlist.
- **Interview subgraph:** Schedule → Collect feedback → Decide.

A parent workflow can compose these subgraphs. This improves modularity, reuse and maintainability, and can help structure multi-agent systems.

Subgraphs in practice: a `screen_resume` subgraph can be developed and tested independently, then called by an `overall_hiring` graph. Decide which fields should be shared with the parent (`candidate_id`, `screening_result`) and which should remain local to the subgraph (temporary parsing details).

When not to split: a two-step pipeline rarely benefits from five layers of nested graphs. Use subgraphs when a unit has meaningful internal control flow, testing requirements or reusability.

3.7 Observability - 1:15:53

Observability means being able to explain *what happened* during a run and *why*. You may want to inspect model inputs/outputs, node execution order, durations, errors, state changes and human decisions.

LangSmith is a tracing/monitoring product in the LangChain ecosystem. LangGraph's graph structure makes the orchestration path explicit and can be traced, helping debug complex runs. Ordinary Python or external actions still need proper instrumentation to be fully visible.

Practical question: When a candidate was rejected, can you identify the decision node, model/tool inputs, state at that point and error history?

What to inspect when debugging

A useful trace answers: *Which node ran? What input did it receive? Which tools did it call? Which branch did it choose? What changed in state? How long did it take?* An LLM answer alone often cannot explain a workflow failure.

Useful metrics: node latency, tool error rate, retry count, tokens per run, number of approval interruptions, and successful completion rate. Avoid logging secrets, private candidate details or unnecessary personal data. Use trace metadata rather than treating tracing as a free pass to store everything.

4. Side-by-side decision table

Aspect	Simple LangChain-style pipeline	LangGraph
Main strength	Reusable LLM building blocks; straightforward steps	Flexible workflow orchestration
Typical flow	Mostly sequential	Sequential, branching and cyclic
State between nodes	Often passed/managed by application	Explicit shared state schema

Aspect	Simple LangChain-style pipeline	LangGraph
Pause/resume	Usually requires custom orchestration	Supported with persistence + interrupts
Human approval	Application-managed	First-class interrupt/resume pattern
Fault recovery	Often application-managed	Checkpoints and retry capabilities
Reuse	Reusable runnables/functions	Subgraphs as reusable workflows
Debugging	LangSmith plus app logs	Graph/path/state visibility with tracing
Good examples	Basic RAG; summarizer; extraction	Hiring workflow; research agent; multi-stage approvals

> **Do not treat this as "LangChain vs LangGraph: choose only one."** LangChain provides models, tools, retrievers and integrations; LangGraph orchestrates them. In today's ecosystem, higher-level LangChain agents can already leverage LangGraph under the hood.

Decision guide: which should I use?

Your requirement	Practical starting point
Call a model with a reusable prompt	LangChain model + prompt
Build a simple RAG question-answering pipeline	LangChain retrieval + runnable pipeline
Build a standard tool-using agent quickly	High-level LangChain agent (LangGraph-backed)
Custom approval, branching or looping workflow	LangGraph `StateGraph`
Multiple specialized teams/subprocesses	LangGraph with subgraphs
Multi-day workflow with saved progress	LangGraph + durable checkpointer + external triggers

Rule of thumb: start with the simplest architecture that expresses the required behavior, and introduce graph-level orchestration when you need explicit control over it.

5. LangGraph vocabulary worth memorizing

- **Node:** A unit of work, typically a Python function that reads state and returns updates.
- **Edge:** The transition from one node to another.
- **Conditional edge:** Routes execution based on a decision or state.
- **State:** A typed record of the workflow's current data.
- **Reducer:** A rule for combining updates to a state field, useful with parallel updates.
- **Checkpointer:** Stores execution state by thread so it can be inspected and resumed.
- **Thread ID:** Identifier used to retrieve the correct conversation/workflow state.
- **Interrupt + resume:** Pause until external/human input becomes available.
- **Subgraph:** A graph used as part of a larger graph.
- **LangSmith:** Observability, tracing, and evaluation platform.

6. Two practice implementations

6.1 Plain Python first: see the workflow problem

```
# Conceptual business logic; helper functions are placeholders.
state = {"jd": "", "approved": False, "revision": 0}
while not state["approved"] and state["revision"] < 3:
    state["jd"] = generate_or_revise_jd(state["jd"])
    state["approved"] = request_human_approval(state["jd"])
```

```
    state["revision"] += 1

if state["approved"]:
    post_job(state["jd"])
else:
    escalate_to_recruiter(state["jd"])
```

This illustrates branching and loops, but a production deployment still needs durable state, safe external actions and explicit pause/resume behavior.

6.2 A complete LangGraph router that runs without an API key

```
# pip install -U langgraph
from typing import TypedDict
from langgraph.graph import StateGraph, START, END

class RequestState(TypedDict):
    question: str
    category: str
    reply: str

def classify(state: RequestState):
    topic = "billing" if "refund" in state["question"].lower() else "general"
    return {"category": topic}

def route(state: RequestState):
    return state["category"]

def billing(state: RequestState):
    return {"reply": "I can explain the refund process."}

def general(state: RequestState):
    return {"reply": "I can help with general questions."}

builder = StateGraph(RequestState)
builder.add_node("classify", classify)
builder.add_node("billing", billing)
builder.add_node("general", general)
builder.add_edge(START, "classify")
builder.add_conditional_edges("classify", route,
                              {"billing": "billing", "general": "general"})
builder.add_edge("billing", END)
builder.add_edge("general", END)
app = builder.compile()
print(app.invoke({"question": "How do I get a refund?", "category": "", "reply": ""}))
```

Read the code from bottom to top: declare the state, implement nodes, specify transitions, compile, then invoke. The router itself is deterministic so that the graph logic is easy to observe before adding an LLM.

Mini project idea

Extend the router with a `needs_human` field, a review node and a checkpointer. Test two cases: (1) a normal question ends immediately; (2) a sensitive request pauses until approval. Capture the state before and after resumption.

7. Revision checklist

- ☐ Explain **LangChain's main building blocks** with one example each.
- ☐ Draw the **hiring workflow**, marking each branch, loop and pause.
- ☐ Explain the difference between **state** and a **checkpoint**.

- ☐ Give an example of **event-driven resume**.
- ☐ Explain why **fault recovery** requires durable storage in production.
- ☐ Describe **human-in-the-loop** interrupt and resume.
- ☐ Explain **subgraphs** and **LangSmith observability**.
- ☐ Know when to use **LangChain, LangGraph, or both**.

8. Revisit these parts of the lesson

- 15:40 - Hiring workflow: pause and map every step to a node.
- 27:36 - Control flow: sketch both approval routes.
- 39:16 - State: list which fields must survive between steps.
- 49:50 - Event-driven work: identify the external trigger.
- 55:55 - Failure recovery: distinguish retry and resume.
- 1:02:26 - Human review: say what `thread_id` identifies.
- 1:08:54 - Nested workflows: isolate one reusable subgraph.
- 1:15:53 - Observability: propose three debugging signals.

Check your understanding

1. Is `StateGraph` a replacement for all LangChain model integrations? Why not?
2. If two branches append to the same `events` list, how are updates combined?
3. Why can an in-memory checkpointer be unsuitable for a multi-day process?
4. What makes an external API call safe during retry or resume?
5. Which pieces belong to the workflow engine and which to your webhook/service infrastructure?

Further reading

- CampusX lesson - LangChain vs LangGraph
- LangChain and LangGraph learning hub
- LangGraph Graph API: nodes, state and edges
- LangGraph persistence: checkpointers and stores
- LangGraph human interrupts and resume
- LangGraph thinking guide: business process to graph